Layer Normalization (LayerNorm) in Transformers

Core idea: For each token independently, LayerNorm normalizes its hidden/feature dimensions, then applies learnable scale (gamma) and shift (beta). Its purpose is to stabilize activations and make optimization more stable.

1. Why LayerNorm Is Needed

Transformer layers repeatedly transform token representations through attention and feed-forward networks. Activation values can change substantially in scale as they pass through layers. LayerNorm keeps each token representation in a controlled statistical form, helping training stability, optimization, and gradient flow.

Important: LayerNorm does not permanently force values into a fixed range. Learnable gamma and beta restore flexibility by allowing scaling and shifting after normalization.

2. What Exactly Is Normalized?

If one token has d_model features, for example [x1, x2, ..., xd], LayerNorm calculates the mean and variance over those d features for that token.

Rule to remember: LayerNorm normalizes across the feature/embedding (hidden) dimension, independently for every token.

For a sequence, Token 1 gets its own statistics, Token 2 gets its own statistics, and so on. LayerNorm does not combine all tokens and calculate one sentence-level mean.

3. Mathematics

For x = [x1, ..., xd]:

Mean: mu = (1/d) * sum(xi)

Variance: sigma^2 = (1/d) * sum((xi - mu)^2)

Normalize: x_hat_i = (xi - mu) / sqrt(sigma^2 + epsilon)

Learnable transformation: yi = gamma_i * x_hat_i + beta_i

Complete formula: y = gamma * (x - mu) / sqrt(sigma^2 + epsilon) + beta

epsilon is a very small constant used for numerical stability, especially to avoid division by zero when variance is zero or extremely small.

4. Why gamma and beta Are Learnable

Normalization changes the mean and scale of the representation. If the network needs a feature to have a different scale or offset, the learned parameters can adapt it. gamma controls scale; beta controls shift. This gives the model normalization benefits without making the transformation rigid.

5. Numerical Example

For x = [2, 4, 6]: mean = 4. Variance = ((2-4)^2 + (4-4)^2 + (6-4)^2) / 3 = 8/3. The normalized values are approximately [-1.225, 0, 1.225]. These are then transformed using the learned gamma and beta.

6. LayerNorm Inside a Transformer

A Transformer layer contains major sublayers such as Multi-Head Attention and a Feed-Forward Network (FFN), with residual connections around them. The exact position of LayerNorm depends on the architecture.

Original Transformer / Post-LN:

y = LayerNorm(x + Attention(x))

and similarly: y = LayerNorm(x + FFN(x))

Modern Pre-LN style:

y = x + Attention(LayerNorm(x))

and similarly: y = x + FFN(LayerNorm(x))

7. Pre-LN vs Post-LN

Post-LN: sublayer -> residual addition -> LayerNorm. This is the style used by the original Transformer.

Pre-LN: LayerNorm -> sublayer -> residual addition. It is common in modern deep Transformers because the direct residual pathway generally makes optimization and gradient flow more stable.

The key difference is simply where normalization occurs relative to the sublayer and residual addition.

8. Residual Connections and LayerNorm

A residual connection forms z = x + Sublayer(x). It gives the original representation a direct path forward and helps gradients propagate through deep networks. LayerNorm has a different role: it stabilizes the resulting representation.

Remember: residual connection = shortcut for information/gradients; LayerNorm = stabilization of representations.

9. LayerNorm vs BatchNorm

LayerNorm	BatchNorm
Normalizes features within each individual token/example.	Uses statistics involving the batch dimension.
Does not require other examples in the batch.	Training statistics depend on the batch.
Natural fit for variable sequence lengths and single-example/autoregressive inference.	Batch-dependent behavior is less natural for sequence modeling.

10. Why LayerNorm Fits Transformers

Transformers commonly handle variable sequence lengths, padding, different batch sizes, and autoregressive generation where inference may proceed token by token. LayerNorm computes statistics from each token's own features, so it does not depend on other examples in the batch.

11. What LayerNorm Does NOT Do

It does not normalize the whole sentence as one vector. It does not calculate attention weights. It does not provide positional information. It does not replace positional mechanisms such as sinusoidal positional encoding, learned positional embeddings, or RoPE.

12. Relationship With Attention and FFN

Self-attention computes relationships between tokens using Q, K, and V. LayerNorm is a separate operation placed before or after the attention sublayer depending on the architecture. The FFN is another separate sublayer and can similarly be preceded by or followed by LayerNorm.

Typical FFN: FFN(x) = W2 * sigma(W1 * x + b1) + b2

13. Does LayerNorm Destroy Information?

LayerNorm changes the mean and scale of the current representation, but it is not simply throwing away semantic information. The learned gamma and beta provide flexibility, and surrounding Transformer layers learn representations that work effectively with normalization. Residual connections also preserve a direct path for the original representation.

14. Compact Mental Model

Think of every token as carrying d_model numbers. LayerNorm looks only at that token's feature vector, calculates its mean and variance, normalizes those features, and then learns the useful scale and offset.

Token -> feature vector -> mean/variance -> normalize -> gamma/beta -> stabilized representation

15. Interview-Ready Answers

What is LayerNorm? It normalizes the feature dimensions of each token independently using that token's mean and variance, followed by learnable scale and shift parameters.

Which dimension is normalized? The hidden/embedding dimension for each token independently, not across tokens or the batch.

Why LayerNorm instead of BatchNorm? It does not depend on batch statistics, so it fits variable-length sequences, varying batch sizes, and autoregressive inference naturally.

What are gamma and beta? gamma is learnable scale and beta is learnable shift.

Why epsilon? To prevent numerical instability such as division by zero when variance is zero or very small.

Pre-LN vs Post-LN? Pre-LN uses x + Sublayer(LayerNorm(x)); Post-LN uses LayerNorm(x + Sublayer(x)). Pre-LN is widely used in modern deep Transformers for more stable optimization.

16. Final Revision Box

LayerNorm: normalize features across one token.
Purpose: stabilize activations and improve optimization.
Formula: y = gamma * (x - mu) / sqrt(sigma^2 + epsilon) + beta.
Residual: provides an information/gradient shortcut.
Post-LN: normalize after residual addition.
Pre-LN: normalize before the sublayer.
Not its job: attention calculation or positional encoding.